

Corso di Laurea Magistrale in Ingegneria Informatica

CORSO DI ALGORITMI E STRUTTURE DATI

Prof. ROBERTO PIETRANTUONO

Prima prova - 17/01/2022

Indicazioni

Si consegna un file in **formato editabile (.txt, .docx, .rtf, etc.)** nominandolo "*CognomeNome*", in cui è riportata l'implementazione (nel linguaggio scelto) seguita da una indicazione della complessità temporale dell'algoritmo implementato (complessità nel caso peggiore, è sufficiente il limite superiore $O(f(n))$). Se si utilizzano librerie di cui non si conosce la complessità, lo si indichi nella spiegazione (ad esempio, "la complessità è $O(n \log n)$ al netto della complessità dell'algoritmo x , che è non nota"). Se si utilizza la randomizzazione, si indichi anche il tempo di esecuzione atteso.

PROBLEMA 1

Si richiede di analizzare un particolare algoritmo di ordinamento. L'algoritmo elabora una sequenza di n interi distinti scambiando due elementi adiacenti finché la sequenza non viene ordinata in ordine crescente. Per la sequenza di input: 91054, l'algoritmo produce l'output 01459.

Bisogna determinare quante operazioni di scambio sono necessarie a quest'algoritmo per ordinare una determinata sequenza di input.

INPUT

L'input contiene diversi casi di test. Ogni test case inizia con una riga che contiene un singolo intero $n < 500.000$ — la lunghezza della sequenza di input. Ciascuna delle seguenti n righe contiene un singolo intero $0 \leq a[i] \leq 999$. L'immissione termina con una sequenza di lunghezza $n = 0$, che chiaramente non deve essere elaborata.

OUTPUT

Per ogni sequenza di input, il programma stampa una singola riga contenente il numero minimo di operazioni di scambio necessarie per ordinare la sequenza di input data.

Sample Input

```
5
9
1
0
5
4
3
1
2
3
0
```

Sample Output

```
6
0
```

PROBLEMA 2

Si scriva un algoritmo per ordinare una sequenza tramite sequenze di operazioni di “flip”. In particolare, si supponga che gli elementi siano disposti lungo una pila di dimensione n , con l'elemento più in alto della pila avente posizione n e l'elemento più in basso posizione 1. Si vuole ordinare la pila in modo che il numero più grande sia in basso (posizione 1) e il numero più piccolo in alto (posizione n).

La singola operazione di flip consiste in un capovolgimento degli elementi: data una posizione p a cui il flip è applicato, l'operazione capovolge la sotto-pila costituita dagli elementi con posizione maggiori o uguali a p .

Ad esempio, considera le tre pile di seguito:

8	7	2
4	6	5
6	4	8
7	8	4
5	5	6
2	2	7

La pila a sinistra può essere trasformata nella pila al centro tramite l'operazione *flip(3)* (ossia capovolgendo gli elementi da 7 in su). La pila centrale può essere trasformata nella pila di destra tramite *flip(1)* (ossia capovolgendo l'intera pila).

INPUT

L'input è costituito da una sequenza di pile. Ogni pila è composta da 1 a 30 elementi compresi tra 1 e 100. Ciascuna pila viene specificata su una singola riga di input, con l'elemento superiore (posizione n) che appare per primo sulla riga, e quello inferiore (posizione 1) che appare per ultimo.

OUTPUT

Per ogni pila, il programma stampi la pila originale su una riga, seguita da una sequenza di operazioni flip che ordinano la pila in modo che l'elemento più grande sia in basso (posizione 1, quindi ultimo della riga) e il più piccolo in alto (posizione n , quindi primo sulla riga). La sequenza di flip per ogni pila dovrebbe terminare con uno 0, che indica che non sono necessari altri flip.

Sample Input

```
1 2 3 4 5
5 4 3 2 1
5 1 2 3 4
```

Sample Output

```
1 2 3 4 5
0
5 4 3 2 1
1 0
5 1 2 3 4
1 2 0
```